Decisiones de diseño · SmartLibrary

Taller práctico Bloque 7 · Diseño de Software · Caso de uso «Registrar préstamo» (POST /api/prestamos)

1. **Tres capas con dependencia en un solo sentido.** Controller → Service → Repository. Cada capa cambia por una sola razón (HTTP, reglas de negocio o persistencia): alta cohesión y bajo acoplamiento. El Controller nunca usa un Repository directamente.
2. **Se elimina GestorPrestamos.** Era un «objeto Dios» con listas en memoria que además coordinaba el caso de uso. Su almacenamiento pasó a los repositorios y su coordinación a PrestamoService; la entrada HTTP, que no existía, quedó en PrestamoController.
3. **PrestamoService es el Controlador GRASP.** Es el primer objeto fuera de la interfaz que recibe la operación del sistema registrarPrestamo y la coordina. El @RestController es presentación: si la petición llegara por otra vía, el controlador GRASP sería el mismo. PrestamoService también es Creador de Prestamo, porque tiene estudiante, libro y fecha.
4. **La regla de disponibilidad vive en la capa de negocio.** El Service la verifica antes de escribir (R1, R5) y Libro.marcarPrestado() protege el estado del libro (Experto en información). Se eliminó setDisponible() público para que nadie pueda saltarse R1.
5. **Persistencia en MongoDB con referencias.** Colecciones estudiantes, libros y prestamos. Prestamo guarda referencias (@DocumentReference, solo los ids) y no copias embebidas, porque estudiante y libro tienen ciclo de vida propio y aparecen en muchos préstamos (asociación 0..* a 1). Por eso en UML son asociaciones y no composiciones.
6. **Consistencia sin transacción entre documentos.** MongoDB es atómico por documento y, sin replica set, no ofrece transacciones entre documentos. Todas las validaciones ocurren antes de escribir (R4). Luego se guarda primero el préstamo y después el libro: si lo segundo fallara, la consulta existsByLibroIdAndFechaDevolucionIsNull (R5) sigue impidiendo un segundo préstamo. Con MongoDB Atlas (replica set) se podría añadir MongoTransactionManager.
7. **Repositorios como interfaces.** Extienden MongoRepository; Spring Data genera la implementación en tiempo de ejecución. Por eso el diagrama muestra generalización entre interfaces y no realización.
8. **DTO y errores explícitos.** PrestamoRequest trae solo los ids validados con @NotBlank (400 si faltan) y PrestamoResponse evita exponer el documento. Dos excepciones de negocio se traducen en un @RestControllerAdvice a 404 (recurso inexistente) y 409 (libro no disponible), sin condicionales de negocio en el Controller.
9. **Relaciones UML justificadas.** Entre capas: asociaciones dirigidas 1 a 1, porque cada capa guarda la siguiente como atributo inyectado. Dominio: Prestamo 0..* → 1 Estudiante y Prestamo 0..* → 1 Libro. Dependencias «use», «create» y «throw» para usos puntuales sin atributo.
10. **Trazabilidad verificada.** Cada regla tiene su clase, su operación y su prueba: 12 pruebas automáticas (Controller y Service) y peticiones HTTP reales contra MongoDB (201, 409, 404, 404, 200 y 400).